

ISBN erkennt gängige Fehler!

Berechnung aus $z_n \dots z_i \dots z_0$ (enthält Prüfziffern) mit Gewichten g_i

$$\sum_{i=0}^n g_i z_i \mod m = 0$$

- 1.) Erkennung **einzelner falscher Ziffern** ist garantiert, wenn alle Gewichte g_i teilerfremd zu m sind (d.h. $\text{ggT}(g_i, m) = 1$)
 - 2.) Erkennung der **Vertauschung** zweier Ziffern z_i und z_k ist garantiert, wenn $g_i - g_k$ teilerfremd zu m ist.
- Primzahlen als Modul m sinnvoll

ISBN $g_1=10, g_2=9, \dots, g_{10}=1$

1.) z.B. z_2 falsch eingegeben:

$$g_1 z_1 + g_2 z_2 + \dots + g_{10} z_{10} \equiv 0 \pmod{m}$$

$$\Leftrightarrow g_1 z_1 + g_3 z_3 + g_4 z_4 + \dots + g_{10} z_{10} \stackrel{(*)}{\equiv} -g_2 z_2$$

Wird nun anstatt z_2 ein $a \neq z_2$ eingegeben, so schlägt der Algo. Alarm:

$$g_1 z_1 + g_2 a + g_3 z_3 + \dots + g_{10} z_{10} \stackrel{(*)}{\equiv} g_2 a - g_2 z_2 \neq 0$$

Ann: $g_2 a - g_2 z_2 \equiv 0 \Rightarrow g_2 a \equiv g_2 z_2 \xrightarrow{g_2^{-1}} a \equiv z_2 \checkmark$

existiert da g_2 teilerfremd zu m !

2.) z.B. z_1 & z_2 vertauscht mit $z_1 \neq z_2$

$$g_1 z_1 + g_2 z_2 + g_3 z_3 + \dots + g_{10} z_{10} \equiv 0 \Rightarrow g_3 z_3 + \dots + g_{10} z_{10} \stackrel{(**)}{\equiv} -g_1 z_1 - g_2 z_2$$

Wird nun z_1 & z_2 vertauscht, so schlägt der Algo. Alarm:

$$\begin{aligned} g_1 z_2 + g_2 z_1 + g_3 z_3 + \dots + g_{10} z_{10} &\stackrel{(**)}{=} g_1 z_2 + g_2 z_1 - g_1 z_1 - g_2 z_2 \\ &= (g_1 - g_2) z_2 + (g_2 - g_1) z_1 \\ &= (g_1 - g_2) (z_2 + z_1) \neq 0 \end{aligned}$$

Ann: $(g_1 - g_2) (z_2 + z_1) \equiv 0$. $g_1 - g_2$ teilerfremd zu m , d.h. $\exists (g_1 - g_2)^{-1}$

$$\stackrel{(g_1 - g_2)^{-1}}{\Rightarrow} z_2 + z_1 \equiv 0$$

$$\Rightarrow z_2 \equiv -z_1 \not\Leftarrow \text{da } z_i \in \{0, 1, \dots, 9\} \text{ \& } z_2 \neq z_1$$

Inverse: z.B. $5 \cdot \underline{9} \equiv 1 \pmod{11}$ d.h. $5^{-1} = 9$

Division/Modulo mit großen Zahlen? Schriftliches Dividieren!

$$21352965 : 13 = 1642997 \text{ R } 4$$

$\begin{array}{r} 13 \\ \underline{21352965} \\ -83 \end{array}$

$21352965 / 13 \xrightarrow{\text{div}}$ $21352965 \% 13 \xrightarrow{\text{mod}}$

4

Große Zahlen in C? z.B. mit Arrays!

unsigned long long a = $2^{64} - 1$; $(\underbrace{11 \dots 1}_\text{nur 64 Bits})_2$

Zahlen mit bis zu $2^{64} - 1$ Bits kann man mit Arrays darstellen:

int a[] = {0, ..., 1, 1, 0, 1, 1, 0, 0, 1, 1, 0};

$2^{64} - 1$ Stellen!! (In Kryptographie werden nur ca. 2048 Bits gebraucht!)

Die Operationen +, -, ·, / müssen dann selbst implementiert werden!

Man orientiert sich dazu an dem schriftlichen Verfahren aus der Schule:

z.B.

$$\begin{array}{r} a \quad \{1, 1, 1, 0\} \\ b \quad + \{0, 1, 1, 0\} \\ \hline \text{übertrag:} \quad 1 \ 1 \ 1 \ 0 \ 0 \\ e = \{1, 0, 1, 0, 0\} \end{array}$$

$$\begin{array}{r} \{1, 1, 1, 0\} \cdot \{0, 1, 1\} \\ \hline \{0000\} \\ \{1110\} \\ + \quad 11110 \\ \hline e = \{1, 0, 1, 0, 1, 0\} \end{array}$$

Summe mit Indexverschieb.